

Chương 25: Kỹ Thuật Trái Phẳng Cây (Euler Tour on Tree)

[THUẬT NGỮ KHOA HỌC] Kỹ Thuật Trái Phẳng Cây & Thứ Tự Euler Tour

Từ nguyên học: Xuất phát từ chu trình Euler trên đồ thị, kỹ thuật Euler Tour trên cây được Robert Tarjan và các nhà khoa học máy tính ứng dụng để chuyển đổi cấu trúc phân cấp phức tạp thành mảng tuyến tính một chiều.

Mô hình trực giác: Cây là cấu trúc dữ liệu phân nhánh 2D khó xử lý đồng thời nhiều nút con. Khi áp dụng phép duyệt DFS và ghi nhận thời điểm bước vào $tin[u]$ cùng thời điểm rời khỏi $tout[u]$, toàn bộ cây con của đỉnh u sẽ được 'gói gọn' hoàn hảo vào một đoạn chỉ số liên tiếp $[tin[u], tout[u]]$. Mọi thao tác trên cây con lập tức quy về thao tác trên đoạn của mảng 1D.

25.1 Cốt Lõi Thuật Toán: Mảng Thời Gian tin và $tout$

Khi duyệt DFS cây có gốc (mặc định gốc 1), ta duy trì một bộ đếm thời gian toàn cục $timer = 0$:

- Thời điểm vào ($tin[u]$):** Ngay khi hàm DFS bắt đầu thăm đỉnh u , ta tăng $timer$ lên 1 và lưu $tin[u] = ++timer$.
- Duyệt con:** Lần lượt gọi đệ quy DFS thăm tất cả các đỉnh con trực tiếp v của u (bỏ qua đỉnh cha p).
- Thời điểm ra ($tout[u]$):** Sau khi toàn bộ các nhánh con của u đã được duyệt xong xuôi, ta chốt mốc thời gian $tout[u] = timer$.

[TIP] Tính chất vàng của đoạn $[tin[u], tout[u]]$

Mọi đỉnh v nằm trong cây con của đỉnh u đều được duyệt **sau khi vào u** và **trước khi rời khỏi u** . Do đó, tập hợp các đỉnh trong cây con của u có chỉ số thời điểm vào nằm trọn trong đoạn liên tục $[tin[u], tout[u]]$. Đặc biệt, kích thước cây con của u chính bằng:

$$subtree_size[u] = tout[u] - tin[u] + 1$$

25.2 Kiểm Tra Quan Hệ Tổ Tiên - Con Cháu Trong $O(1)$

Cho hai đỉnh bất kỳ u và v . Đỉnh u là tổ tiên của đỉnh v (kể cả trường hợp $u = v$) khi và chỉ khi khoảng thời gian sống của v nằm trọn trong khoảng thời gian sống của u :

$$tin[u] \leq tin[v] \quad \text{và} \quad tout[u] \geq tout[v]$$

Nhờ tính chất này, ta có thể trả lời câu hỏi “Đỉnh u có phải là tổ tiên của v không?” chỉ trong một phép so sánh $O(1)$ mà không cần duyệt ngược lên gốc!

25.3 Traversal Trọng Số & Truy Vấn Tổng Cây Con Tĩnh

Nếu mỗi đỉnh u mang trọng số W_u , ta tạo mảng traversal F kích thước N :

$$F[\text{tin}[u]] = W_u$$

Khi đó, tổng trọng số các đỉnh trong cây con gốc u chính là tổng của mảng F trên đoạn $[\text{tin}[u], \text{tout}[u]]$:

$$\sum_{v \in \text{subtree}(u)} W_v = \sum_{i = \text{tin}[u]}^{\text{tout}[u]} F[i]$$

Dùng mảng cộng dồn tiền tố $\text{pref}[i] = \text{pref}[i-1] + F[i]$, truy vấn tổng cây con được tính trong $O(1)$:

$$\text{subtree_sum}(u) = \text{pref}[\text{tout}[u]] - \text{pref}[\text{tin}[u] - 1]$$

[MẪU XẼ CÚ PHÁP C++14]	
<pre>void dfs(int u, int p) { tin[u] = ++timer_cnt; for (int v : adj[u]) if (v != p) dfs(v, u); tout[u] = timer_cnt; }</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>tin[u] = ++timer_cnt;</code>	Ghi nhận mốc thời gian bắt đầu bước vào cây con của u .
<code>for (int v : adj[u])</code>	Duyệt qua các đỉnh kề của u theo thứ tự xác định.
<code>if (v != p) dfs(v, u);</code>	Ngăn chặn quay ngược lên cha, đệ quy xuống từng nhánh con.
<code>tout[u] = timer_cnt;</code>	Chốt mốc thời gian rời khỏi u sau khi tất cả các con đã hoàn tất.

25.4 Bảng Chạy Bàn Trên Giấy: Cây 4 Đỉnh Với Cạnh $(1, 2), (1, 3), (2, 4)$

Trọng số $W = [10, 20, 30, 40]$. Cây có gốc tại 1.

Đỉnh u	$\text{tin}[u]$	$\text{tout}[u]$	Đoạn $[\text{tin}, \text{tout}]$	Cây con gồm	Tổng trọng số cây con
1	1	4	[1, 4]	{1, 2, 4, 3}	$10 + 20 + 40 + 30 = 100$
2	2	3	[2, 3]	{2, 4}	$20 + 40 = 60$
4	3	3	[3, 3]	{4}	40
3	4	4	[4, 4]	{3}	30

Mảng traversal: $F = [W_1 = 10, W_2 = 20, W_4 = 40, W_3 = 30]$. Cây con của đỉnh 2 nằm trên đoạn $[2, 3]$, tổng là $F[2] + F[3] = 20 + 40 = 60$.

25.5 Bài Tập Tiêu Chuẩn

Bài toán: Bài 25.1 - Kỹ Thuật Trái Phải Cây: Thứ Tự Duyệt Euler Tour & Tổng Cây Con

(Nguồn: Chuyên khảo CP C++14 - Cấu trúc dữ liệu nâng cao)

Cho một đồ thị cây gồm N đỉnh ($1 \leq N \leq 30$) có gốc tại đỉnh 1. Mỗi đỉnh u có trọng số W_u ($1 \leq W_u \leq 1000$). Các cạnh vô hướng được cho dưới dạng danh sách $N - 1$ cặp đỉnh (u, v) . Để thử tự duyệt Euler Tour là tiền định và thống nhất khi tính toán trên giấy:

- Tại mỗi đỉnh, danh sách các đỉnh kề được duyệt theo thứ tự số hiệu đỉnh tăng dần.
- Duyệt DFS từ gốc 1 với biến đếm thời gian `timer` khởi tạo bằng 0: gán `tin[u] = ++timer` khi vào, duyệt con, và gán `tout[u] = timer` khi kết thúc.
- Mảng trái phằng $F[tin[u]] = W_u$.
- Cho đỉnh truy vấn u_1 và cặp đỉnh (u_2, v_2) .

Em hãy tính toán và in ra trên một dòng 4 số nguyên cách nhau bởi dấu cách:

1. `tin[u1]`: Mốc thời gian vào của đỉnh u_1 .
2. `tout[u1]`: Mốc thời gian ra của đỉnh u_1 .
3. `sum_u1`: Tổng trọng số của tất cả các đỉnh thuộc cây con gốc u_1 .
4. `is_anc`: Bằng `1` nếu u_2 là tổ tiên của v_2 ($\text{tin}[u_2] \leq \text{tin}[v_2] \wedge \text{tout}[u_2] \geq \text{tout}[v_2]$), ngược lại in `0`.

Đầu vào (Input):

- Dòng 1: Ghi 4 số nguyên N, u_1, u_2, v_2 .
- Dòng 2: Ghi N số nguyên dương $W_1, W_2, \dots, W_N$.
- $N - 1$ dòng tiếp theo: Mỗi dòng ghi 2 số u, v mô tả một cạnh của cây.

Đầu ra (Output): In ra 4 số nguyên `tin[u1]` `tout[u1]` `sum_u1` `is_anc` trên một dòng.

Ví dụ mẫu:

Đầu vào (Input):

```

4 2 1 4
10 20 30 40
1 2
1 3
2 4

```

Đầu ra (Output):

2 3 60 1

Thủ thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	4 2 1 4 10 20 30 40 1 2 1 3 2 4	[.....]

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
2	Warm-up	<pre> 5 3 2 5 5 10 15 20 25 1 2 2 3 3 4 4 5 </pre>	[.....]
3	Warm-up	<pre> 5 1 1 3 100 10 20 30 40 1 2 1 3 1 4 1 5 </pre>	[.....]
4	Edge case	<pre> 1 1 1 1 50 </pre>	[.....]
5	Edge case	<pre> 2 2 2 1 10 20 1 2 </pre>	[.....]
6	Edge case	<pre> 3 2 2 2 5 15 25 1 2 1 3 </pre>	[.....]
7	Trick case	<pre> 6 2 2 6 1 1 1 1 1 1 1 2 1 3 2 4 2 5 3 6 </pre>	[.....]
8	Trick case	<pre> 6 6 3 6 2 4 6 8 10 12 1 2 2 3 3 4 4 5 5 6 </pre>	[.....]
9	Mental challenge	<pre> 7 3 2 7 1 2 3 4 5 6 7 1 2 1 3 2 4 2 5 3 6 3 7 </pre>	[.....]
10	Mental challenge	<pre> 8 4 1 8 3 1 4 1 5 9 2 6 1 2 1 3 2 4 2 5 3 6 4 7 4 8 </pre>	[.....]

Chương 26: Kỹ Thuật Nhảy Nhị Phân (Binary Lifting)

[THUẬT NGỮ KHOA HỌC] Kỹ Thuật Nhảy Nhị Phân & Đồ Thị Hàm

Từ nguyên học: Kỹ thuật nhân đôi khoảng cách (Doubling) bắt nguồn từ các thuật toán tính lũy thừa ma trận và tìm kiếm nhị phân, sau đó được tổng quát hóa thành Binary Lifting để duyệt cấu trúc đồ thị hàm và cây.

Mô hình trực giác: Thay vì bước từng bước đơn lẻ $O(K)$ lặp đi lặp lại rất chậm chạp, ta chuẩn bị sẵn các bước nhảy có kích thước là lũy thừa của 2: bước 1, bước 2, bước 4, bước 8, bước 16... Do mọi số nguyên K đều có thể phân tích duy nhất thành tổng các lũy thừa của 2 dưới hệ nhị phân, ta chỉ cần tối đa $\log_2(K)$ cú nhảy để chạm tới đích chính xác tuyệt đối.

26.1 Đồ Thị Hàm (Functional Graphs) & Phép Hợp Hàm $f^{\{(K)\}}(x)$

Trong một đồ thị hàm có N đỉnh $\{1, 2, \dots, N\}$, mỗi đỉnh x có đúng một cung đi ra tới $f(x) = \text{nxt}[x]$. Nếu xuất phát từ đỉnh ban đầu S và di chuyển liên tiếp K bước theo quy tắc $x \rightarrow f(x)$, ta muốn tìm đỉnh đích:

$$\text{dest} = f^{(K)}(S) = \underbrace{f(f(\dots f(S)\dots))}_{K \text{ lần}}$$

Khi K lên tới 10^9 hoặc 10^{18} , mô phỏng vòng lặp tuần tự sẽ dẫn đến TLE ngay lập tức. Kỹ thuật Binary Lifting cho phép giải quyết bài toán chỉ trong $O(\log K)$ phép tính.

26.2 Xây Dựng Bảng Nhảy Nhị Phân (Sparse Table of Jumps)

Ta định nghĩa bảng nhảy hai chiều $\text{up}[j][x]$:

- $\text{up}[0][x] = f(x) = \text{nxt}[x]$: Đỉnh đến được sau khi nhảy $2^0 = 1$ bước từ x .
- $\text{up}[j][x]$: Đỉnh đến được sau khi nhảy 2^j bước từ x .

Công thức truy hồi nhân đôi (Doubling Recurrence): Vì $2^j = 2^{j-1} + 2^{j-1}$, nên để nhảy 2^j bước từ x , ta chỉ cần:

- Nhảy 2^{j-1} bước từ x đến đỉnh trung gian $y = \text{up}[j-1][x]$.
- Từ y , tiếp tục nhảy thêm 2^{j-1} bước nữa để đến $\text{up}[j-1][y]$.

$$\text{up}[j][x] = \text{up}[j-1][\text{up}[j-1][x]]$$

Chi phí tiền xử lý toàn bộ bảng $\text{up}[0..29][1..N]$ chỉ mất $O(N \log K)$ thời gian và bộ nhớ.

26.3 Thuật Toán Nhảy Nhị Phân Trong $O(\log K)$

Mọi số nguyên K đều biểu diễn được dưới dạng nhị phân: $K = \sum_{j=0}^M b_j \cdot 2^j$ với $b_j \in \{0, 1\}$. Để thực hiện K bước nhảy từ đỉnh S :

```

cur = S
for j từ 0 đến M:
    nếu bit thứ j của K bật (tức (K >> j) & 1 == 1):
        cur = up[j][cur]

```

[LÝ THUYẾT] Trục giác nhị phân

Ví dụ $K = 13 = 1101_2 = 2^3 + 2^2 + 2^0 = 8 + 4 + 1$. Từ S , ta chỉ cần thực hiện 3 cú nhảy: $S \xrightarrow{2^0=1} x_1 \xrightarrow{2^2=4} x_2 \xrightarrow{2^3=8} x_3 = \text{đích}$.

[MÔ XẼ CÚ PHÁP C++14]

```
for (int j = 0; j < 30; ++j) { if ((k >> j) & 1) cur = up[j][cur]; }
```

Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>int j = 0; j < 30; ++j</code>	Duyệt qua các trọng số lũy thừa 2 từ 2^0 đến 2^{29} (vượt quá 10^9).
<code>(k >> j) & 1</code>	Trích xuất giá trị bit thứ j trong biểu diễn nhị phân của số bước nhảy K.
<code>cur = up[j][cur];</code>	Cú nhảy nhị phân kích thước 2^j đưa con trỏ tức thời tới đỉnh mới trong $O(1)$.

26.4 Bảng Chạy Bàn Trên Giấy: Đồ Thị 4 Đỉnh Với $f = [2, 3, 4, 1]$

Mảng $f = [2, 3, 4, 1]$ là một chu trình 4 đỉnh $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$. Ta xây dựng bảng $up[j][x]$ cho các lũy thừa $2^0 = 1, 2^1 = 2, 2^2 = 4$:

Lũy thừa 2^j	Độ dài bước	$x = 1$	$x = 2$	$x = 3$	$x = 4$	Công thức truy hồi
$j = 0$	1 bước	<code>f(1) = 2</code>	<code>f(2) = 3</code>	<code>f(3) = 4</code>	<code>f(4) = 1</code>	<code>up[0][x] = f(x)</code>
$j = 1$	2 bước	<code>up[0][2] = 3</code>	<code>up[0][3] = 4</code>	<code>up[0][4] = 1</code>	<code>up[0][1] = 2</code>	<code>up[1][x] = up[0][up[0][x]]</code>
$j = 2$	4 bước	<code>up[1][3] = 1</code>	<code>up[1][4] = 2</code>	<code>up[1][1] = 3</code>	<code>up[1][2] = 4</code>	<code>up[2][x] = up[1][up[1][x]]</code>

Nếu muốn tìm $f^{\{(5)\}}(1)$ với $K = 5 = 2^2 + 2^0 = 4 + 1$:

- Bit 0 bật ($2^0 = 1$): $cur = up[0][1] = 2$.
- Bit 1 tắt ($2^1 = 2$): bỏ qua.
- Bit 2 bật ($2^2 = 4$): $cur = up[2][2] = 2$.

Kết quả: sau 5 bước từ đỉnh 1 ta dừng tại đỉnh 2!

26.5 Bài Tập Tiêu Chuẩn

Bài toán: Bài 26.1 - Kỹ Thuật Nhảy Nhị Phân:
Nhảy Trên Đồ Thi Hàm

(Nguồn: Chuyên khảo CP C++14 - Cấu trúc dữ liệu nâng cao)

Cho một đồ thị hàm gồm N đỉnh ($1 \leq N \leq 30$). Mỗi đỉnh i ($1 \leq i \leq N$) có một con trỏ trỏ tới đỉnh tiếp theo $\text{nxt}[i]$ ($1 \leq \text{nxt}[i] \leq N$) và mang giá trị trọng số $A[i]$ ($1 \leq A[i] \leq 1000$). Cho đỉnh xuất phát S ($1 \leq S \leq N$) và số bước nhảy K ($0 \leq K \leq 10^9$). Sử dụng bảng nhảy nhị phân `up[j][x]`, hãy xác định:

- Đỉnh dừng lại sau đúng K bước nhảy: $\text{dest} = f^{(K)}(S)$.
- Đỉnh dừng lại ở mốc nửa chặng đường: $\text{mid} = f^{\lfloor \frac{K}{2} \rfloor}(S)$.
- Trong số của đỉnh đích: $A[\text{dest}]$.

Đầu vào (Input):

- Dòng 1: Ghi 3 số nguyên N, S, K .
- Dòng 2: Ghi N số nguyên $\text{nxt}_1, \text{nxt}_2, \dots, \text{nxt}_N$.
- Dòng 3: Ghi N số nguyên $A_1, A_2, \dots, A_N$.

Đầu ra (Output): In ra 3 số nguyên `dest mid A[dest]` cách nhau bởi dấu cách trên một dòng.

Ví dụ mẫu:

Đầu vào (Input):

```
4 1 5
2 3 4 1
10 20 30 40
```

Đầu ra (Output):

2 3 20

Thủ thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	4 1 5 2 3 4 1 10 20 30 40	[.....]
2	Warm-up	5 1 3 2 3 4 5 5 1 2 3 4 5	[.....]
3	Warm-up	3 2 7 2 3 1 100 200 300	[.....]
4	Edge case	4 2 0 2 3 4 1 10 20 30 40	[.....]
5	Edge case	1 1 1000000 1 99	[.....]
6	Edge case	3 1 1 3 1 2 7 8 9	[.....]

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
7	Trick case	6 1 10 2 3 4 5 6 4 5 10 15 20 25 30	[.....]
8	Trick case	4 1 1000000000 2 1 4 3 11 22 33 44	[.....]
9	Mental challenge	7 3 6 3 4 5 6 7 1 2 7 6 5 4 3 2 1	[.....]
10	Mental challenge	8 1 8 2 4 1 3 6 8 5 7 12 24 36 48 60 72 84 96	[.....]

Chương 27: Tổ Tiên Chung Gần Nhất (Lowest Common Ancestor - LCA)

[THUẬT NGỮ KHOA HỌC] Tổ Tiên Chung Gần Nhất (LCA) Trên Cây

Từ nguyên học: Bài toán LCA được Aho, Hopcroft và Ullman chính thức đặt ra năm 1973 và phát triển rực rỡ qua công trình của Harel & Tarjan (1984), Bender & Farach-Colton (2000).

Mô hình trực giác: Trên một cái cây có gốc, khi hai người đứng ở hai đỉnh bất kỳ u và v cùng nhìn ngược lên cội nguồn (gốc cây), các nhánh cội nguồn của họ cuối cùng sẽ giao nhau tại một nhánh chung đầu tiên. Đỉnh giao nhau đó chính là Tổ tiên chung gần nhất $LCA(u, v)$. LCA là 'ngã ba đường' phân định ranh giới giữa hai nhánh và là chìa khóa vàng để tính khoảng cách giữa mọi cặp đỉnh trên cây.

27.1 Cấu Trúc Bảng Tổ Tiên Nhị Phân $up[k][u]$

Cho cây N đỉnh có gốc tại đỉnh 1.

- Mỗi đỉnh u có độ sâu $depth[u]$ (quy ước gốc $depth[1] = 1$).
- Đỉnh cha trực tiếp của u là $parent[u]$ (gốc 1 có $parent[1] = 1$).
- Bảng $up[k][u]$ lưu tổ tiên thứ 2^k của đỉnh u :

$$up[0][u] = parent[u]$$

$$up[k][u] = up[k-1][up[k-1][u]] \quad (k = 1, 2, \dots, 19)$$

Bảng này được tiền xử lý hoàn tất trong $O(N \log N)$ sau đúng một lần duyệt DFS.

27.2 Thuật Toán Tìm LCA Qua Hai Giai Đoạn

Để tìm $LCA(u, v)$, giả sử không mất tính tổng quát rằng $depth[u] \geq depth[v]$:

Giai đoạn 1: Cân bằng độ sâu

- Tính chênh lệch độ sâu: $\Delta = depth[u] - depth[v]$.
- Dùng kỹ thuật Binary Lifting nâng đỉnh u lên Δ bậc để đưa u về cùng độ sâu với v .
- Nếu sau khi nâng mà $u == v$: Ta kết luận ngay $LCA(u, v) = v$ (vì v chính là tổ tiên trực tiếp của u).

Giai đoạn 2: Cùng nhảy lên ngã ba

- Duyệt số mũ k giảm dần từ $\log_2 N$ về 0.
- Tại mỗi k , nếu $up[k][u] \neq up[k][v]$, ta cùng nâng cả hai: $u = up[k][u]$ và $v = up[k][v]$.
- Khi dừng lại, u và v nằm ngay dưới ngã ba chung. Do đó:

$$LCA = up[0][u]$$

27.3 Công Thức Tính Khoảng Cách Trên Cây

Đường đi ngắn nhất giữa hai đỉnh u và v trên cây luôn đi từ u lên $LCA(u, v)$, rồi từ $LCA(u, v)$ đi xuống v . Do đó, khoảng cách (số cạnh) giữa hai đỉnh được tính tức thời qua công thức:

$$\text{dist}(u, v) = \text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[LCA(u, v)]$$

[MẪU XẼ CÚ PHÁP C++14]	
<pre>for (int k = MAX_LOG - 1; k ≥ 0; --k) { if (up[k][u] ≠ up[k][v]) { u = up[k][u]; v = up[k][v]; } } return up[0][u];</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<pre>for (int k = MAX_LOG - 1; k ≥ 0; --k)</pre>	Duyệt bước nhảy giảm dần từ 2^{19} xuống 2^0 theo nguyên lý tham lam nhị phân.
<pre>if (up[k][u] ≠ up[k][v])</pre>	Chỉ nhảy khi hai đỉnh chưa vượt qua đỉnh tổ tiên chung để không bị vượt ngã ba.
<pre>u = up[k][u]; v = up[k][v];</pre>	Đồng thời nâng cả hai đỉnh lên độ cao mới bảo toàn sự cân bằng độ sâu.
<pre>return up[0][u];</pre>	Sau vòng lặp, u và v đứng ngay dưới LCA, bước cha trực tiếp $up[0][u]$ chính là kết quả.

27.4 Bảng Chạy Bàn Trên Giấy: Cây 5 Đỉnh Gốc 1

Cạnh: (1, 2), (1, 3), (2, 4), (2, 5). Độ sâu: $\text{depth}[1] = 1, \text{depth}[2] = 2, \text{depth}[3] = 2, \text{depth}[4] = 3, \text{depth}[5] = 3$.

Cặp truy vấn (u, v)	depth[u], depth[v]	Cân bằng độ sâu	Nhảy đồng thời	LCA	Khoảng cách dist(u, v)
(4, 5)	3, 3	Cùng depth = 3	$up[0][4] = 2$ $= up[0][5]$ $= 2$	2	$3 + 3 - 2(2)$ $= 2$
(4, 3)	3, 2	$u = up[0][4]$ $= 2$	$up[0][2] = 1$ $= up[0][3]$ $= 1$	1	$3 + 2 - 2(1)$ $= 3$
(1, 5)	1, 3	$v = up[1][5]$ $= 1 = u$	Dừng ngay: $u = v$	1	$1 + 3 - 2(1)$ $= 2$

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
4	Edge case	1 1 1	[.....]
5	Edge case	2 1 2 1 2	[.....]
6	Edge case	4 3 3 1 2 2 3 3 4	[.....]
7	Trick case	6 2 6 1 2 1 3 1 4 1 5 1 6	[.....]
8	Trick case	6 1 6 1 2 2 3 3 4 4 5 5 6	[.....]
9	Mental challenge	7 4 7 1 2 1 3 2 4 2 5 3 6 3 7	[.....]
10	Mental challenge	8 7 8 1 2 1 3 2 4 2 5 5 6 6 7 3 8	[.....]

Chương 28: Cây Chỉ Số Nhị Phân (Fenwick Tree / BIT)

[THUẬT NGỮ KHOA HỌC] Cây Chỉ Số Nhị Phân (Binary Indexed Tree / Fenwick Tree)

Từ nguyên học: Được Peter Fenwick công bố năm 1994 nhằm mục đích nén dữ liệu, cấu trúc cây chỉ số nhị phân nhanh chóng trở thành một trong những cấu trúc dữ liệu tinh giản và hiệu quả nhất của lập trình thi đấu.

Mô hình trực giác: Thay vì lưu toàn bộ tổng cộng dồn (chỉ truy vấn nhanh $O(1)$ nhưng cập nhật cực chậm $O(N)$), hoặc lưu mảng thô (cập nhật $O(1)$ nhưng tính tổng $O(N)$), Cây Fenwick phân chia mảng thành các đoạn lũy thừa của 2. Mỗi ô nhớ i chịu trách nhiệm quản lý đúng một đoạn có độ dài bằng bit 1 nhỏ nhất của i ($\text{lowbit}(i) = i \& -i$). Cả thao tác cập nhật điểm lẫn tính tổng đoạn đều đạt tốc độ kinh ngạc $O(\log N)$ với chỉ 3 dòng mã.

28.1 Phép Toán Lowbit Thần Thánh: $i \& (-i)$

Dưới hệ biểu diễn số bù 2 (Two's Complement), số đối $-i$ được tạo thành bằng cách đảo toàn bộ các bit của i rồi cộng thêm 1:

- Phép phủ định bit $\sim i$ biến bit 1 thấp nhất của i thành 0, và biến toàn bộ các bit 0 bên phải nó thành 1.
- Khi cộng thêm 1, các bit 1 này bật thành 0 và bit tại vị trí bit 1 thấp nhất lại bật thành 1.
- Do đó, phép toán logic $i \& (-i)$ cô lập chính xác lũy thừa của 2 tương ứng với bit 1 nhỏ nhất của i :

$$\text{lowbit}(i) = i \wedge (-i)$$

[TIP] Ví dụ giá trị lowbit

- $i = 6 = 110_2 \rightarrow \text{lowbit}(6) = 2 = 010_2$.
- $i = 12 = 1100_2 \rightarrow \text{lowbit}(12) = 4 = 0100_2$.
- $i = 8 = 1000_2 \rightarrow \text{lowbit}(8) = 8 = 1000_2$.

28.2 Trách Nhiệm Của Nút & Hai Thao Tác Cơ Bản

Mỗi phần tử $\text{tree}[i]$ trong mảng Fenwick (1-indexed) lưu tổng của đúng $\text{lowbit}(i)$ phần tử kết thúc tại i :

$$\text{tree}[i] = \sum_{k=i-\text{lowbit}(i)+1}^i A[k]$$

1. Cập nhật điểm: $\text{add}(i, \text{delta})$

- Khi giá trị tại vị trí i tăng thêm Δ , ta phải cộng Δ vào tất cả các ô $\text{tree}[k]$ chịu trách nhiệm bao trùm i .

2. Truy vấn tiền tố: $\text{query}(i)$

- Để tính tổng tiền tố $\sum_{k=1}^i A[k]$, ta cộng dồn giá trị tại $\text{tree}[i]$, rồi nhảy lùi về phần tử trước đoạn trách nhiệm:

- Các chỉ số k này được duyệt bằng cách nhảy tiến:

$$k \rightarrow k + \text{lowbit}(k)$$

- Độ phức tạp: $O(\log N)$.

$$k \rightarrow k - \text{lowbit}(k)$$

- Lặp lại cho đến khi $k = 0$.
- Tổng đoạn $[L, R] = \text{query}(R) - \text{query}(L - 1)$.
- Độ phức tạp: $O(\log N)$.

28.3 Ứng Dụng Đếm Số Cặp Nghịch Thế (Inversions)

Một cặp nghịch thế là cặp chỉ số (i, j) sao cho $i < j$ nhưng $A[i] > A[j]$. Thuật toán đếm nghịch thế bằng Fenwick Tree:

- Rời rạc hóa giá trị của mảng A về khoảng thứ hạng $[1, K]$ ($K \leq N$).
- Khởi tạo một Cây Fenwick rỗng biểu diễn tần số xuất hiện.
- Duyệt lần lượt từng phần tử $A[i]$ từ trái sang phải:
 - Số phần tử đã xuất hiện trước đó nhưng lớn hơn $A[i]$ chính là:

$$\text{inv} += \text{query}(K) - \text{query}(\text{rank}(A[i]))$$

- Thêm phần tử hiện tại vào cây tần số: `"add"("rank"(A[i]), 1)`.

[MỖ XÈ CÚ PHÁP C++14]

```
for (; idx ≤ n; idx += idx & (-idx)) tree[idx] += delta;
```

Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>for (; idx ≤ n;</code>	Duyệt tiến qua các nút tổ tiên trong cây Fenwick cho tới biên kích thước n .
<code>idx += idx & (-idx)</code>	Cộng thêm $\text{lowbit}(\text{idx})$ để nhảy tới nút cha bao trùm đoạn trách nhiệm tiếp theo.
<code>tree[idx] += delta;</code>	Cập nhật giá trị dồn tích lũy vào nút tương ứng.

28.4 Bảng Chạy Bàn Trên Giấy: Mảng $A = [3, 1, 2, 5, 4]$ Với $N = 5$

Phần tử $A[i]$	Rank	Số phần tử lớn hơn đã gặp	Tổng nghịch thế dồn	Cập nhật Fenwick
$A[1] = 3$	3	$\text{query}(5) - \text{query}(3) = 0 - 0 = 0$	0	<code>add(3, 1)</code>
$A[2] = 1$	1	$\text{query}(5) - \text{query}(1) = 1 - 0 = 1$	1 (cặp (3, 1))	<code>add(1, 1)</code>
$A[3] = 2$	2	$\text{query}(5) - \text{query}(2) = 2 - 1 = 1$	2 (thêm (3, 2))	<code>add(2, 1)</code>

Phần tử $A[i]$	Rank	Số phần tử lớn hơn đã gặp	Tổng nghịch thể dồn	Cập nhật Fenwick
$A[4] = 5$	5	$query(5) - query(5) = 3 - 3 = 0$	2	$add(5, 1)$
$A[5] = 4$	4	$query(5) - query(4) = 4 - 3 = 1$	*3 (thêm (5, 4))*	$add(4, 1)$

Tổng số cặp nghịch thể là 3: (3, 1), (3, 2), (5, 4). Mảng ban đầu có tổng đoạn $[2, 4]$ là $A[2] + A[3] + A[4] = 1 + 2 + 5 = 8$.

28.5 Bài Tập Tiêu Chuẩn

Bài toán: Bài 28.1 - Cây Chi Số Nhị Phân: Cập Nhật Điểm, Truy Vấn Đoạn & Đếm Nghịch Thể

(Nguồn: Chuyên khảo CP C++14 - Cấu trúc dữ liệu nâng cao)

Cho một dãy gồm N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq N \leq 30, 1 \leq A_i \leq 100$). Sử dụng cấu trúc Cây Fenwick (BIT), em hãy thực hiện các công việc sau:

- Đếm tổng số cặp nghịch thể trong dãy ban đầu: số cặp (i, j) thỏa mãn $1 \leq i < j \leq N$ và $A_i > A_j$.
- Xây dựng Cây Fenwick lưu trữ mảng ban đầu, tính tổng các phần tử trong đoạn $[L, R]$:

$$S_1 = \sum_{i=L}^R A_i$$

- Thực hiện thao tác cập nhật điểm: cộng thêm giá trị V vào vị trí P ($A_P \rightarrow A_P + V$).
- Tính lại tổng đoạn $[L, R]$ sau khi đã cập nhật:

$$S_2 = \sum_{i=L}^R A_i$$

Đầu vào (Input):

- Dòng 1: Ghi 5 số nguyên N, P, V, L, R .
- Dòng 2: Ghi N số nguyên $A_1, A_2, \dots, A_N$.

Đầu ra (Output): In ra 3 số nguyên `inv S1 S2` cách nhau bởi dấu cách trên một dòng.

Ví dụ mẫu:

Đầu vào (Input):

53524

31254

Đầu ra (Output):

3813

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	53524 31254	

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
2	Warm-up	4 2 10 1 4 1 2 3 4	[.....]
3	Warm-up	4 1 2 2 3 4 3 2 1	[.....]
4	Edge case	1 1 5 1 1 10	[.....]
5	Edge case	2 1 100 2 2 20 30	[.....]
6	Edge case	3 2 0 1 3 5 5 5	[.....]
7	Trick case	6 4 7 2 5 2 2 2 2 2 2	[.....]
8	Trick case	5 3 -10 1 5 10 20 30 40 50	[.....]
9	Mental challenge	7 5 3 3 6 5 2 6 1 3 7 4	[.....]
10	Mental challenge	8 2 8 1 8 3 1 4 1 5 9 2 6	[.....]

Chương 29: Thuật Toán Băm Xâu (Polynomial Rolling Hashing)

[THUẬT NGỮ KHOA HỌC] Thuật Toán Băm Xâu Đa Thức (Polynomial Rolling Hash)

Từ nguyên học: Được Michael Rabin và Richard Karp công bố năm 1987, kỹ thuật băm xâu lăn (Rolling Hash) đã đưa bài toán tìm kiếm mẫu chuỗi về các phép số học modulo kỳ diệu.

Mô hình trực giác: Một chuỗi ký tự bản chất là một số nguyên cực lớn được viết trong hệ cơ số BASE. Bằng cách lấy đồng dư với một số nguyên tố MOD đủ lớn, ta thu gọn chuỗi có độ dài tùy ý thành một số nguyên duy nhất (mã băm). Nhờ nguyên lý tiền tố, mã băm của bất kỳ đoạn con $[L, R]$ nào cũng được tính trong $O(1)$ bằng đúng một phép trừ và nhân lũy thừa, mở ra khả năng so sánh chuỗi tương đương tốc độ của phép so sánh số nguyên.

29.1 Định Nghĩa Hàm Băm Đa Thức Tiền Tố

Cho chuỗi ký tự S có độ dài N , các ký tự được chuẩn hóa thành số nguyên từ 1 đến 26 ($c - 'a' + 1$). Chọn cơ số $BASE = 31$ (hoặc 311) lớn hơn kích thước bảng chữ cái và modulo nguyên tố $MOD = 10^9 + 7$. Mảng băm tiền tố H (1-indexed) được xây dựng theo quy tắc nhân cuộn:

$$H[0] = 0$$

$$H[i] = (H[i-1] \cdot BASE + S[i]) \bmod MOD$$

Mảng lũy thừa $power[k] = BASE^k \bmod MOD$ được tiền xử lý trước.

29.2 Trích Xuất Mã Băm Đoạn Con $[L, R]$ Trong $O(1)$

Mã băm của đoạn con $S[L...R]$ tương ứng với việc dịch chuyển đoạn đó về cuối và loại bỏ phần tiền tố thừa $S[1...L-1]$:

$$\text{hash}(L, R) = (H[R] - H[L-1] \cdot BASE^{R-L+1}) \bmod MOD$$

Để xử lý kết quả âm trong C++, ta luôn cộng thêm MOD trước khi lấy dư:

$$\text{hash}(L, R) = ((H[R] - H[L-1] \cdot power[R-L+1]) \bmod MOD + MOD) \bmod MOD$$

29.3 Ứng Dụng Thuật Toán Rabin-Karp & Kiểm Tra Palindrome

1. Thuật toán Rabin-Karp

2. Kiểm Tra Đối Xứng (Palindrome)

- Tính trước mã băm của xâu mẫu P : $\text{hash_P} = \text{hash}(P)$.
- Trượt một cửa sổ có độ dài $|P|$ trên chuỗi S .
- Với mỗi vị trí bắt đầu i , so sánh mã băm $\text{hash}(i, i + |P| - 1)$ với hash_P trong $O(1)$.
- Tổng thời gian quét tìm kiếm: $O(|S| + |P|)$.

- Xây dựng mảng băm tiền tố xuôi H của xâu S và mảng băm tiền tố ngược RH của xâu đảo rev_S .
- Đoạn con $S[L...R]$ là đối xứng khi và chỉ khi:

$$\text{hash_xuôi}(L, R) == \text{hash_ngược}(L, R)$$
- Thao tác kiểm tra tính đối xứng của bất kỳ đoạn con nào chỉ mất $O(1)$ thời gian.

[MỖ XÈ CÚ PHÁP C++14]

```
long long res = (h[r] - h[l - 1] * power[r - l + 1]) % MOD; if (res < 0) res
+= MOD;
```

Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>h[r] -</code>	Lấy mã băm tiền tố từ đầu đến r .
<code>h[l - 1] * power[r - l + 1]</code>	Nhân dời cơ số để cân bằng bậc lũy thừa với tiền tố $l-1$.
<code>% MOD;</code>	Phép chia lấy dư bảo toàn tính số học trong phạm vi nguyên.
<code>if (res < 0) res += MOD;</code>	Bù lại phần dư âm do phép trừ trong C++.

29.4 Bảng Chạy Bàn Trên Giấy: Chuỗi $S = \text{abacaba}$, Mẫu $P = \text{aba}$

Độ dài $N = 7, M = 3$. Các ký tự: $a = 1, b = 2, c = 3$. Hàm băm với $\text{BASE} = 31, \text{MOD} = 10^9 + 7$:
 $\text{hash}(P) = (1 \cdot 31^2 + 2 \cdot 31 + 1) = 961 + 62 + 1 = 1024$.

Vị trí i	Đoạn con $S[i..i+2]$	Mã băm trích xuất $\text{hash}(i, i+2)$	So khớp với $\text{hash}(P) = 1024$	Kết luận
1	aba	1024	<code>1024 == 1024</code> (Khớp)	Xuất hiện lần 1 tại pos = 1
2	bac	1986	<code>1986 != 1024</code>	Không khớp
3	aca	1055	<code>1055 != 1024</code>	Không khớp
4	cab	2947	<code>2947 != 1024</code>	Không khớp
5	aba	1024	<code>1024 == 1024</code> (Khớp)	*Xuất hiện lần 2 tại pos = 5*

Đoạn $[1, 7]$ là chuỗi abacaba có mã băm xuôi trùng với mã băm ngược $\rightarrow$ là Palindrome!

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
5	Edge case	abc abcde 1 3	[.....]
6	Edge case	abcdef cd 3 3	[.....]
7	Trick case	aaaaaa aa 1 6	[.....]
8	Trick case	noonnight night 1 4	[.....]
9	Mental challenge	radarad rad 1 5	[.....]
10	Mental challenge	abcabcba abc 3 6	[.....]

Chương 30: Thao Tác Bit & Quy Hoạch Động Mặt Nạ (Bitmask DP)

[THUẬT NGỮ KHOA HỌC] Thao Tác Bit & Quy Hoạch Động Trạng Thái Mặt Nạ (Bitmask DP)

Từ nguyên học: Được Michael Held và Richard Karp phát triển năm 1962 trong thuật toán Bellman-Held-Karp giải bài toán Người du lịch TSP thời gian hàm mũ $O(2^N * N^2)$.

Mô hình trực giác: Một tập hợp con các phần tử có thể được mã hóa hoàn hảo thành một chuỗi nhị phân (Mặt nạ - Bitmask): bit thứ i bằng 1 nghĩa là phần tử i có mặt trong tập, bằng 0 nghĩa là vắng mặt. Bằng cách dùng một số nguyên biểu diễn trạng thái tập hợp, ta có thể áp dụng Quy hoạch động để duyệt qua toàn bộ 2^N tập con với tốc độ phần cứng của thanh ghi CPU.

30.1 Bảng Cấu Kẹt Các Phép Toán Bit Cốt Lõi

Cho một mặt nạ `mask` và phần tử thứ i (0-indexed):

Thao tác cơ bản trên bit

- Kiểm tra bit:** `(mask >> i) & 1` (trả về 1 nếu phần tử i thuộc tập).
- Bật bit:** `mask | (1 << i)` (thêm phần tử i vào tập).
- Tắt bit:** `mask & ~(1 << i)` (loại phần tử i khỏi tập).
- Đảo bit:** `mask ^ (1 << i)` (đổi trạng thái có mặt/vắng mặt).

Hàm nội tại GCC & Duyệt tập con

- `__builtin_popcount(mask)`: Đếm số lượng phần tử trong tập (số bit 1).
- `__builtin_ctz(mask)`: Đếm số bit 0 ở cuối (vị trí phần tử nhỏ nhất).
- Duyệt toàn bộ tập con thực sự của `mask`:

```
for (int s = mask; s > 0; s = (s - 1) & mask)
```

30.2 Bài Toán Người Du Lịch (TSP) & Thiết Kế Trạng Thái DP

Cho đồ thị có hướng N đỉnh với ma trận chi phí $C[u][v]$ ($0 \leq u, v < N$).

- Không gian trạng thái: `dp[mask][u]` là chi phí nhỏ nhất để xuất phát từ đỉnh 0, đi qua tất cả các đỉnh có bit 1 trong `"mask"`, và hiện đang dừng tại đỉnh u .
- Trạng thái cơ sở: `dp[1][0] = 0` (tập đỉnh chỉ gồm đỉnh 0, đang đứng ở 0), các ô khác khởi tạo bằng $+\infty$.
- Công thức chuyển trạng thái: Với mỗi đỉnh v chưa được thăm trong `"mask"` (`!((mask >> v) & 1)`):

$$dp[mask | (1 \ll v)][v] = \min(dp[mask | (1 \ll v)][v], dp[mask][u] + C[u][v])$$

- Chi phí đường đi Hamilton mở qua tất cả N đỉnh:

$$\text{min_path} = \min_{u=0}^{N-1} dp[(1 \ll N) - 1][u]$$

- Chi phí chu trình Hamilton TSP đóng quay về đỉnh 0:

$$\text{min_tsp} = \min_{u=0}^{N-1} (\text{dp}[(1 \ll N) - 1][u] + C[u][0])$$

[MẪU XẺ CÚ PHÁP C++14]	
<pre>int next_mask = mask (1 << v); dp[next_mask][v] = std::min(dp[next_mask][v], dp[mask][u] + cost[u][v]);</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>mask (1 << v)</code>	Bật bit thứ v để ghi nhận đỉnh v đã được bổ sung vào hành trình.
<code>dp[next_mask][v]</code>	Trạng thái mới: tập đỉnh đã thăm là next_mask và đang đứng tại đỉnh v.
<code>dp[mask][u] + cost[u][v]</code>	Chi phí tích lũy tại u cộng thêm chi phí di chuyển cung cạnh (u, v).
<code>std::min(...)</code>	Lựa chọn quyết định tối ưu trong nguyên lý Bellman.

30.3 Bảng Chạy Bàn Trên Giấy: Đồ Thị 3 Đỉnh $N = 3$

Mã trận chi phí: $C = \begin{pmatrix} 0 & 10 & 20 \\ 10 & 0 & 15 \\ 20 & 15 & 0 \end{pmatrix}.$

mask	Các đỉnh đã thăm	Đỉnh hiện tại u	dp[mask][u]	Chuyển tiếp khả dĩ sang v
1 (001)	{0}	0	0 (Cơ sở)	v=1: dp[3][1] = 0+10 = 10; v=2: dp[5][2] = 0+20 = 20
3 (011)	{0, 1}	1	10	v=2: dp[7][2] = min(inf, 10+15) = 25
5 (101)	{0, 2}	2	20	v=1: dp[7][1] = min(inf, 20+15) = 35
7 (111)	{0, 1, 2}	1	35	Đường đi: 0 → 2 → 1, quay về 0: 35 + 10 = 45
7 (111)	{0, 1, 2}	2	25	*Đường đi: 0 → 1 → 2 (min_path = 25)*, về 0: 25 + 20 = 45

Kết quả: min_path = 25, min_tsp = 45.

30.4 Bài Tập Tiêu Chuẩn

Bài toán: Bài 30.1 - Thao Tác Bit & Quy Hoạch

(Nguồn: Chuyên khảo CP C++14 - Cấu trúc dữ liệu nâng cao)

Động Trạng Thái Mặt Nạ: Bài Toán TSP

Cho một đồ thị có hướng gồm N đỉnh ($2 \leq N \leq 8$), các đỉnh được đánh số từ 0 đến $N - 1$. Ma trận chi phí hai chiều $C[N][N]$ cho biết chi phí di chuyển $C[u][v]$ từ đỉnh u đến đỉnh v ($0 \leq C[u][v] \leq 100$, $C[u][u] = 0$). Sử dụng kỹ thuật Quy hoạch động mặt nạ trạng thái Bitmask DP, em hãy tìm:

1. **min_path**: Chi phí nhỏ nhất của đường đi Hamilton mở xuất phát từ đỉnh 0, đi qua tất cả N đỉnh đúng một lần và kết thúc ở bất kỳ đỉnh nào.
2. **min_tsp**: Chi phí nhỏ nhất của chu trình Hamilton TSP xuất phát từ đỉnh 0, đi qua tất cả N đỉnh đúng một lần và quay trở về đỉnh 0.

Đầu vào (Input):

- Dòng 1: Ghi số nguyên N .
- N dòng tiếp theo: Mỗi dòng ghi N số nguyên mô tả hàng thứ u của ma trận chi phí $C[u][v]$.

Đầu ra (Output): In ra 2 số nguyên `min_path min_tsp` cách nhau bởi dấu cách trên một dòng.

Ví dụ mẫu:

Đầu vào (Input):

Đầu ra (Output):

3
0 10 20
10 0 15
20 15 0

25 45

Thủ thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	3 0 10 20 10 0 15 20 15 0	[.....]
2	Warm-up	4 0 2 9 10 1 0 6 4 15 7 0 8 6 3 12 0	[.....]
3	Warm-up	3 0 1 5 5 0 2 1 5 0	[.....]
4	Edge case	2 0 5 7 0	[.....]

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
5	Edge case	<pre> 3 0 1 1 1 0 1 1 1 0 </pre>	
6	Edge case	<pre> 2 0 10 10 0 </pre>	
7	Trick case	<pre> 4 0 1 20 20 20 0 1 20 20 20 0 1 1 20 20 0 </pre>	
8	Trick case	<pre> 4 0 3 4 5 3 0 5 4 4 5 0 3 5 4 3 0 </pre>	
9	Mental challenge	<pre> 5 0 3 1 5 8 3 0 6 7 9 1 6 0 4 2 5 7 4 0 3 8 9 2 3 0 </pre>	
10	Mental challenge	<pre> 6 0 2 4 6 8 10 2 0 3 5 7 9 4 3 0 2 4 6 6 5 2 0 3 5 8 7 4 3 0 2 10 9 6 5 2 0 </pre>	

Bài Tập Tích Hợp Toàn Diện V1

[THUẬT NGỮ KHOA HỌC] Hệ Thống Phối Hợp Kỹ Thuật Nâng Cao V1

Từ nguyên học: Đỉnh cao của lập trình thi đấu hiện đại là khả năng kết hợp nhuần nhuyễn nhiều cấu trúc dữ liệu và giải thuật trong cùng một hệ thống thống nhất.

Mô hình trực giác: Một bài toán cây phức tạp với cập nhật động tài nguyên và truy vấn đường đi không thể giải quyết bằng một kỹ thuật đơn lẻ. Bằng cách kết hợp Euler Tour (để tuyến tính hóa không gian cây con), Cây Fenwick (để cập nhật điểm và truy vấn tổng đoạn trong $O(\log N)$), cùng với Binary Lifting LCA (để định tuyến ngã ba và đo khoảng cách đường đi trong $O(\log N)$), ta kiến tạo một hệ thống vận hành với độ phức tạp tối ưu tuyệt đối.

Kiến Trúc Hệ Thống: Cây Con Động & Nhảy Tổ Tiên

Quy trình giải thuật tích hợp gồm 3 tầng kiến trúc độc lập nhưng kết nối chặt chẽ:

Tầng 1: Tuyến Tính Hóa

- Duyệt DFS từ gốc 1.
- Ghi nhận $tin[u]$, $tout[u]$ và độ sâu $depth[u]$.
- Cây con u trở thành đoạn $[tin[u], tout[u]]$.

Tầng 2: Quản Lý Động BIT

- Mảng trái phải $F[tin[u]] = V_u$.
- Nạp vào Fenwick Tree.
- Cập nhật điểm P :
`add(tin[P], X)`.
- Tổng cây con Q :
`range_sum(tin[Q], tout[Q])`.

Tầng 3: Nhảy Nhị Phân LCA

- Bảng $up[k][u]$ xây từ DFS.
- Cân bằng độ sâu và nhảy đồng thời tìm $LCA(U, V)$.
- Khoảng cách hành quân:
 $depth[U] + depth[V] - 2 \cdot depth[LCA]$.

[MẪU XẺ CÚ PHÁP C++14]

```
bit.add(tin[p], x); long long s_q = bit.range_sum(tin[q], tout[q]); int lca_node = get_lca(u, v);
```

Thành phần cú pháp

Cơ chế ngữ nghĩa & Mục đích

```
bit.add(tin[p], x);
```

Dùng thời điểm vào $tin[p]$ làm chỉ số để cập nhật tài nguyên thành phố P trên Cây Fenwick.

```
bit.range_sum(tin[q], tout[q]);
```

Truy vấn tổng tài nguyên của cây con thành phố Q nhờ đoạn liên tục $[tin, tout]$.

```
get_lca(u, v);
```

Tìm điểm giao lộ ngã ba chung của hành trình quân sự giữa hai thành phố U và V .

Bảng Chạy Bàn Trên Giấy: Cây 5 Đỉnh Gốc 1

Cây: (1, 2), (1, 3), (2, 4), (2, 5). Giá trị ban đầu: $V = [10, 20, 30, 40, 50]$. Truy vấn: Tiếp tế $X = 10$ cho thành phố $P = 3$; tính tổng cây con $Q = 2$; tìm LCA và khoảng cách giữa $U = 4$ và $V = 5$.

Thao tác	Dữ liệu liên quan	Công thức tính toán	Kết quả
DFS Euler Tour	Gốc 1	tin: 1->1, 2->2, 4->3, 5->4, 3->5; tout: 2->4, 1->5	Đoạn cây con Q=2: [2, 4]
Cập nhật Fenwick	$P = 3, X = +10$	<code>bit.add(tin[3]=5, 10)</code> → $V[3]$ thành 40	Tổng toàn cây = $150 + 10 = 160$
Truy vấn cây con Q	$Q = 2$ (đoạn [2, 4])	$V[2] + V[4] + V[5] = 20 + 40 + 50$	$*S_Q = 110*$
LCA & Khoảng cách	$U = 4, V = 5$	$depth[4]=3,$ $depth[5]=3, LCA(4, 5) = 2$	$*LCA = 2, dist = 3 + 3 - 2(2) = 2*$

Kết quả cuối cùng: `160 110 2 2`.

Bài Tập Tiêu Chuẩn

Bài toán: Bài Tập Tích Hợp 4 - Hệ Thống Cây Vương Quốc: Cập Nhật Cây Con Euler Tour + Fenwick Tree & Khoảng Cách LCA

(Nguồn: Chuyên khảo CP C++14 - Tích hợp cấu trúc dữ liệu nâng cao V1)

Một vương quốc gồm N thành phố ($1 \leq N \leq 30$) liên kết dạng cây có gốc tại thủ đô 1. Mỗi thành phố u ban đầu có lượng lương thực tích trữ là V_u ($1 \leq V_u \leq 1000$). Các cạnh vô hướng được cho trong danh sách $N - 1$ cặp đỉnh (các đỉnh kề sắp xếp tăng dần khi DFS). Hệ thống nhận lệnh điều phối tổng hợp gồm:

- Tiếp tế thêm X đơn vị lương thực cho thành phố P ($V_P \rightarrow V_P + X$).
- Kiểm tra tổng lương thực của toàn bộ vương quốc (S_{root}).
- Kiểm tra tổng lương thực trong phân nhánh cây con do thành phố Q chỉ huy (S_Q).
- Tìm thành phố ngã ba trung chuyển $L = LCA(U, V)$ và khoảng cách hành quân $dist(U, V)$ giữa hai đơn vị quân đóng tại U và V .

Em hãy tính toán và in ra trên một dòng 4 số nguyên cách nhau bởi dấu cách: `S_root S_Q LCA_UV dist_UV`

Đầu vào (Input):

- Dòng 1: Ghi 6 số nguyên N, P, X, Q, U, V .
- Dòng 2: Ghi N số nguyên $V_1, V_2, ..., V_N$.
- $N - 1$ dòng tiếp theo: Mỗi dòng ghi 2 số u, v mô tả một cạnh cây.

Đầu ra (Output): In ra 4 số nguyên `S_root S_Q LCA_UV dist_UV` trên một dòng.

Ví dụ mẫu:

Đầu vào (Input):

5 3 10 2 4 5
10 20 30 40 50
1 2
1 3
2 4
2 5

Đầu ra (Output):

160 110 2 2

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	5 3 10 2 4 5 10 20 30 40 50 1 2 1 3 2 4 2 5	[.....]
2	Warm-up	4 1 5 3 1 4 5 10 15 20 1 2 2 3 3 4	[.....]
3	Warm-up	5 4 100 4 2 5 1 2 3 4 5 1 2 1 3 1 4 1 5	[.....]
4	Edge case	1 1 50 1 1 1 100	[.....]
5	Edge case	2 2 0 2 1 2 30 40 1 2	[.....]
6	Edge case	3 2 15 2 2 2 10 20 30 1 2 1 3	[.....]
7	Trick case	6 3 -20 3 4 6 50 40 30 20 10 5 1 2 1 3 2 4 2 5 3 6	[.....]

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
8	Trick case	6 5 10 2 3 6 1 1 1 1 1 1 1 2 1 3 2 4 2 5 3 6	[.....]
9	Mental challenge	7 4 12 3 6 7 1 2 3 4 5 6 7 1 2 1 3 2 4 2 5 3 6 3 7	[.....]
10	Mental challenge	8 6 25 4 7 8 10 20 30 40 50 60 70 80 1 2 1 3 2 4 2 5 3 6 4 7 4 8	[.....]

Phụ Lục: Đáp Án & Bảng Vết Thực Thi 10 Test Bàn Tay

[LÝ THUYẾT] **Hướng Dẫn Đối Chiếu Lời Giải Bằng Bút Chì**

Phụ lục này cung cấp toàn bộ đáp án chính thức và bảng vết thực thi (Execution Trace) từng bước cho tất cả 10 bài test của từng chương trong Tập 4. Sau khi đã tự giải và điền kết quả vào các ô trống bằng bút chì, các em hãy đối chiếu với bảng đáp án dưới đây để phát hiện chính xác bước tư duy nào bị nhầm lẫn.

Lời Giải Chi Tiết Chương 25: Kỹ Thuật Traversal Cây (Euler Tour on Tree)

Lời giải & Đáp án: Bài 25.1 - Kỹ Thuật Traversal Cây: Thử Tự Duyệt Euler Tour & Tổng Cây Con			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	4 2 1 4 10 20 30 40 1 2 1 3 2 4	2 3 60 1	Cây 4 đỉnh rẽ nhánh cơ bản
2	5 3 2 5 5 10 15 20 25 1 2 2 3 3 4 4 5	3 5 60 1	Cây 5 đỉnh dạng đường thẳng, kiểm tra u1 tại giữa
3	5 1 1 3 100 10 20 30 40 1 2 1 3 1 4 1 5	1 5 200 1	Cây 5 đỉnh hình sao tỏa ra từ gốc 1
4	1 1 1 1 50	1 1 50 1	Cây chỉ có 1 đỉnh duy nhất
5	2 2 2 1 10 20 1 2	2 2 20 0	Cây 2 đỉnh, u2 không phải tổ tiên v2
6	3 2 2 2 5 15 25 1 2 1 3	2 2 15 1	Cây 3 đỉnh, u2 và v2 trùng nhau

Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
7	6 2 2 6 1 1 1 1 1 1 1 2 1 3 2 4 2 5 3 6	2 4 3 0	Cây 6 đỉnh với trọng số bằng nhau toàn 1, hai nhánh đối xứng
8	6 6 3 6 2 4 6 8 10 12 1 2 2 3 3 4 4 5 5 6	6 6 12 1	Cây 6 đỉnh, truy vấn đỉnh lá cuối cùng
9	7 3 2 7 1 2 3 4 5 6 7 1 2 1 3 2 4 2 5 3 6 3 7	5 7 16 0	Thử thách nhầm cây nhị phân 7 đỉnh đầy đủ
10	8 4 1 8 3 1 4 1 5 9 2 6 1 2 1 3 2 4 2 5 3 6 4 7 4 8	3 5 9 1	Thử thách nhầm cây 8 đỉnh phân nhánh phức tạp

Bảng Vết Thực Thi Từng Bước (Chương 25)

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	4
-	u1	2
-	u2	1
-	v2	4
2	Euler tour completed	-
-	tin	(0, 1, 2, 4, 3)
-	tout	(0, 4, 3, 4, 3)
3	Subtree sum and ancestor query	-
-	sum_u1	60
-	is_anc	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-

Bước	Mô tả / Biến	Giá trị
-	n	5
-	u1	3
-	u2	2
-	v2	5
2	Euler tour completed	-
-	tin	(0, 1, 2, 3, 4, 5)
-	tout	(0, 5, 5, 5, 5, 5)
3	Subtree sum and ancestor query	-
-	sum_u1	60
-	is_anc	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	5
-	u1	1
-	u2	1
-	v2	3
2	Euler tour completed	-
-	tin	(0, 1, 2, 3, 4, 5)
-	tout	(0, 5, 2, 3, 4, 5)
3	Subtree sum and ancestor query	-
-	sum_u1	200
-	is_anc	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	1
-	u1	1
-	u2	1
-	v2	1
2	Euler tour completed	-
-	tin	(0, 1)
-	tout	(0, 1)
3	Subtree sum and ancestor query	-
-	sum_u1	50
-	is_anc	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	2
-	u1	2
-	u2	2

Bước	Mô tả / Biến	Giá trị
-	v2	1
2	Euler tour completed	-
-	tin	(0, 1, 2)
-	tout	(0, 2, 2)
3	Subtree sum and ancestor query	-
-	sum_u1	20
-	is_anc	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	3
-	u1	2
-	u2	2
-	v2	2
2	Euler tour completed	-
-	tin	(0, 1, 2, 3)
-	tout	(0, 3, 2, 3)
3	Subtree sum and ancestor query	-
-	sum_u1	15
-	is_anc	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	6
-	u1	2
-	u2	2
-	v2	6
2	Euler tour completed	-
-	tin	(0, 1, 2, 5, 3, 4, 6)
-	tout	(0, 6, 4, 6, 3, 4, 6)
3	Subtree sum and ancestor query	-
-	sum_u1	3
-	is_anc	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	6
-	u1	6
-	u2	3
-	v2	6
2	Euler tour completed	-
-	tin	(0, 1, 2, 3, 4, 5, 6)

Bước	Mô tả / Biến	Giá trị
-	tout	(0, 6, 6, 6, 6, 6, 6)
3	Subtree sum and ancestor query	-
-	sum_u1	12
-	is_anc	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	7
-	u1	3
-	u2	2
-	v2	7
2	Euler tour completed	-
-	tin	(0, 1, 2, 5, 3, 4, 6, 7)
-	tout	(0, 7, 4, 7, 3, 4, 6, 7)
3	Subtree sum and ancestor query	-
-	sum_u1	16
-	is_anc	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	8
-	u1	4
-	u2	1
-	v2	8
2	Euler tour completed	-
-	tin	(0, 1, 2, 7, 3, 6, 8, 4, 5)
-	tout	(0, 8, 6, 8, 5, 6, 8, 4, 5)
3	Subtree sum and ancestor query	-
-	sum_u1	9
-	is_anc	1

Lời Giải Chi Tiết Chương 26: Kỹ Thuật Nhảy Nhị Phân (Binary Lifting)

Lời giải & Đáp án: Bài 26.1 - Kỹ Thuật Nhảy Nhị Phân: Nhảy Trên Đồ Thị Hàm			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	4 1 5 2 3 4 1 10 20 30 40	2 3 20	Đồ thị chu trình 4 đỉnh, nhảy K=5 bước
2	5 1 3 2 3 4 5 5 1 2 3 4 5	4 2 4	Đồ thị đường thẳng dẫn tới vòng lặp bản thân, K=3 bước
3	3 2 7 2 3 1 100 200 300	3 2 300	Chu trình 3 đỉnh nhảy K=7 bước
4	4 2 0 2 3 4 1 10 20 30 40	2 2 20	Không nhảy bước nào: K=0
5	1 1 1000000 1 99	1 1 99	Chỉ có 1 đỉnh tự lặp, K rất lớn 1000000
6	3 1 1 3 1 2 7 8 9	3 1 9	Nhảy đúng K=1 bước
7	6 1 10 2 3 4 5 6 4 5 10 15 20 25 30	5 6 25	Đồ thị hình số 6 (cây gậy có vòng chu trình ở đuôi), K=10 bước
8	4 1 1000000000 2 1 4 3 11 22 33 44	1 1 11	Các đỉnh chéo đối xứng, K=1000000000
9	7 3 6 3 4 5 6 7 1 2 7 6 5 4 3 2 1	1 2 7	Thử thách nhầm đồ thị hoán vị 7 đỉnh, K=6 bước
10	8 1 8 2 4 1 3 6 8 5 7 12 24 36 48 60 72 84 96	1 1 12	Thử thách nhầm đồ thị 8 đỉnh với bước nhảy lũy thừa 2: K=8 bước

Bảng Vết Thực Thi Từng Bước (Chương 26)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, s, k	-
-	n	4
-	s	1
-	k	5
2	Construct binary lifting table	-
3	Binary lifting jumps completed	-
-	dest	2

Bước	Mô tả / Biến	Giá trị
-	mid	3
-	a_dest	20

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, s, k	-
-	n	5
-	s	1
-	k	3
2	Construct binary lifting table	-
3	Binary lifting jumps completed	-
-	dest	4
-	mid	2
-	a_dest	4

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, s, k	-
-	n	3
-	s	2
-	k	7
2	Construct binary lifting table	-
3	Binary lifting jumps completed	-
-	dest	3
-	mid	2
-	a_dest	300

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, s, k	-
-	n	4
-	s	2
-	k	0
2	Construct binary lifting table	-
3	Binary lifting jumps completed	-
-	dest	2
-	mid	2
-	a_dest	20

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, s, k	-
-	n	1
-	s	1
-	k	1000000
2	Construct binary lifting table	-

Bước	Mô tả / Biến	Giá trị
3	Binary lifting jumps completed	-
-	dest	1
-	mid	1
-	a_dest	99

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, s, k	-
-	n	3
-	s	1
-	k	1
2	Construct binary lifting table	-
3	Binary lifting jumps completed	-
-	dest	3
-	mid	1
-	a_dest	9

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, s, k	-
-	n	6
-	s	1
-	k	10
2	Construct binary lifting table	-
3	Binary lifting jumps completed	-
-	dest	5
-	mid	6
-	a_dest	25

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, s, k	-
-	n	4
-	s	1
-	k	1000000000
2	Construct binary lifting table	-
3	Binary lifting jumps completed	-
-	dest	1
-	mid	1
-	a_dest	11

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, s, k	-
-	n	7
-	s	3

Bước	Mô tả / Biến	Giá trị
-	k	6
2	Construct binary lifting table	-
3	Binary lifting jumps completed	-
-	dest	1
-	mid	2
-	a_dest	7

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, s, k	-
-	n	8
-	s	1
-	k	8
2	Construct binary lifting table	-
3	Binary lifting jumps completed	-
-	dest	1
-	mid	1
-	a_dest	12

Lời Giải Chi Tiết Chương 27: Tổ Tiên Chung Gần Nhất (LCA on Tree)

Lời giải & Đáp án: Bài 27.1 - Tổ Tiên Chung Gần Nhất: LCA Trên Cây Qua Nhảy Nhị Phân			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	5 4 5 1 2 1 3 2 4 2 5	2 2 2	Cây 5 đỉnh, LCA của 2 đỉnh cùng tầng khác nhánh
2	5 1 5 1 2 2 3 3 4 4 5	1 4 1	Cây 5 đỉnh, quan hệ tổ tiên - con cháu trực hệ
3	6 4 6 1 2 1 3 2 4 3 5 5 6	1 5 1	Cây 6 đỉnh, LCA của 2 lá ở 2 nhánh đối diện qua gốc 1
4	1 1 1	1 0 1	Cây chỉ có 1 đỉnh duy nhất, LCA của chính nó
5	2 1 2 1 2	1 1 1	Cây 2 đỉnh, LCA của gốc và con trực tiếp
6	4 3 3 1 2 2 3 3 4	3 0 3	Truy vấn hai đỉnh trùng nhau u=v trên cây 4 đỉnh
7	6 2 6 1 2 1 3 1 4 1 5 1 6	1 2 1	Cây hình sao, LCA của hai cánh sao luôn là gốc 1
8	6 1 6 1 2 2 3 3 4 4 5 5 6	1 5 1	Cây chuỗi dài 6 đỉnh, khoảng cách tối đa
9	7 4 7 1 2 1 3 2 4 2 5 3 6 3 7	1 4 1	Thử thách nhầm cây nhị phân hoàn hảo 7 đỉnh

Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
10	8 7 8 1 2 1 3 2 4 2 5 5 6 6 7 3 8	1 6 1	Thử thách nhằm cây 8 đỉnh bất đối xứng

Bảng Vết Thực Thi Từng Bước (Chương 27)

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	5
-	q_u	4
-	q_v	5
2	DFS completed and binary lifting table built	-
-	depth	(0, 1, 2, 2, 3, 3)
3	LCA computed	-
-	lca	2
-	dist	2
-	depth_lca	2

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	5
-	q_u	1
-	q_v	5
2	DFS completed and binary lifting table built	-
-	depth	(0, 1, 2, 3, 4, 5)
3	LCA computed	-
-	lca	1
-	dist	4
-	depth_lca	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	6
-	q_u	4
-	q_v	6
2	DFS completed and binary lifting table built	-
-	depth	(0, 1, 2, 2, 3, 3, 4)
3	LCA computed	-
-	lca	1

Bước	Mô tả / Biến	Giá trị
-	dist	5
-	depth_lca	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	1
-	q_u	1
-	q_v	1
2	DFS completed and binary lifting table built	-
-	depth	(0, 1)
3	LCA computed	-
-	lca	1
-	dist	0
-	depth_lca	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	2
-	q_u	1
-	q_v	2
2	DFS completed and binary lifting table built	-
-	depth	(0, 1, 2)
3	LCA computed	-
-	lca	1
-	dist	1
-	depth_lca	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	4
-	q_u	3
-	q_v	3
2	DFS completed and binary lifting table built	-
-	depth	(0, 1, 2, 3, 4)
3	LCA computed	-
-	lca	3
-	dist	0
-	depth_lca	3

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	6

Bước	Mô tả / Biến	Giá trị
-	q_u	2
-	q_v	6
2	DFS completed and binary lifting table built	-
-	depth	(0, 1, 2, 2, 2, 2, 2)
3	LCA computed	-
-	lca	1
-	dist	2
-	depth_lca	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	6
-	q_u	1
-	q_v	6
2	DFS completed and binary lifting table built	-
-	depth	(0, 1, 2, 3, 4, 5, 6)
3	LCA computed	-
-	lca	1
-	dist	5
-	depth_lca	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	7
-	q_u	4
-	q_v	7
2	DFS completed and binary lifting table built	-
-	depth	(0, 1, 2, 2, 3, 3, 3, 3)
3	LCA computed	-
-	lca	1
-	dist	4
-	depth_lca	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and graph	-
-	n	8
-	q_u	7
-	q_v	8
2	DFS completed and binary lifting table built	-
-	depth	(0, 1, 2, 2, 3, 3, 4, 5, 3)
3	LCA computed	-
-	lca	1

Bước	Mô tả / Biến	Giá trị
-	dist	6
-	depth_lca	1

Lời Giải Chi Tiết Chương 28: Cây Chỉ Số Nhị Phân (Fenwick Tree / BIT)

Lời giải & Đáp án: Bài 28.1 - Cây Chỉ Số Nhị Phân: Cập Nhật Điểm, Truy Vấn Đoạn & Đếm Nghịch Thế			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	5 3 5 2 4 3 1 2 5 4	3 8 13	Mảng 5 phần tử cơ bản có nghịch thế
2	4 2 10 1 4 1 2 3 4	0 10 20	Mảng 4 phần tử đã tăng dần (0 nghịch thế)
3	4 1 2 2 3 4 3 2 1	6 5 5	Mảng 4 phần tử giảm dần hoàn toàn
4	1 1 5 1 1 10	0 10 15	Mảng 1 phần tử
5	2 1 100 2 2 20 30	0 30 30	Mảng 2 phần tử, cập nhật điểm ngoài đoạn truy vấn
6	3 2 0 1 3 5 5 5	0 15 15	Cập nhật với giá trị 0 không đổi
7	6 4 7 2 5 2 2 2 2 2	0 8 15	Tất cả các phần tử bằng nhau (không có nghịch thế)
8	5 3 -10 1 5 10 20 30 40 50	0 150 140	Cập nhật số âm làm giảm tổng
9	7 5 3 3 6 5 2 6 1 3 7 4	9 17 20	Thử thách nhầm mảng 7 phần tử hoán vị ngược
10	8 2 8 1 8 3 1 4 1 5 9 2 6	8 31 39	Thử thách nhầm mảng 8 phần tử với nhiều số lặp lại

Bảng Vết Thực Thi Từng Bước (Chương 28)

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, p, v, l, r and array a	-
-	n	5
-	p	3
-	v	5
-	l	2
-	r	4
2	Count inversions completed	-
-	inv_count	3
3	Initial range sum query S1	-
-	s1	8
4	Range sum after point update S2	-
-	s2	13

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, p, v, l, r and array a	-
-	n	4
-	p	2
-	v	10
-	l	1
-	r	4
2	Count inversions completed	-
-	inv_count	0
3	Initial range sum query S1	-
-	s1	10
4	Range sum after point update S2	-
-	s2	20

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, p, v, l, r and array a	-
-	n	4
-	p	1
-	v	2
-	l	2
-	r	3
2	Count inversions completed	-
-	inv_count	6
3	Initial range sum query S1	-
-	s1	5
4	Range sum after point update S2	-
-	s2	5

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, p, v, l, r and array a	-
-	n	1
-	p	1
-	v	5
-	l	1
-	r	1
2	Count inversions completed	-
-	inv_count	0
3	Initial range sum query S1	-
-	s1	10
4	Range sum after point update S2	-
-	s2	15

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, p, v, l, r and array a	-
-	n	2
-	p	1
-	v	100
-	l	2
-	r	2
2	Count inversions completed	-
-	inv_count	0
3	Initial range sum query S1	-
-	s1	30
4	Range sum after point update S2	-
-	s2	30

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, p, v, l, r and array a	-
-	n	3
-	p	2
-	v	0
-	l	1
-	r	3
2	Count inversions completed	-
-	inv_count	0
3	Initial range sum query S1	-
-	s1	15
4	Range sum after point update S2	-
-	s2	15

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, p, v, l, r and array a	-
-	n	6
-	p	4
-	v	7
-	l	2
-	r	5
2	Count inversions completed	-
-	inv_count	0
3	Initial range sum query S1	-
-	s1	8
4	Range sum after point update S2	-
-	s2	15

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, p, v, l, r and array a	-
-	n	5
-	p	3
-	v	-10
-	l	1
-	r	5
2	Count inversions completed	-
-	inv_count	0
3	Initial range sum query S1	-
-	s1	150
4	Range sum after point update S2	-
-	s2	140

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, p, v, l, r and array a	-
-	n	7
-	p	5
-	v	3
-	l	3
-	r	6
2	Count inversions completed	-
-	inv_count	9
3	Initial range sum query S1	-
-	s1	17
4	Range sum after point update S2	-
-	s2	20

Bước	Mô tả / Biến	Giá trị
1	Read inputs n, p, v, l, r and array a	-
-	n	8
-	p	2
-	v	8
-	l	1
-	r	8
2	Count inversions completed	-
-	inv_count	8
3	Initial range sum query S1	-
-	s1	31
4	Range sum after point update S2	-
-	s2	39

Lời Giải Chi Tiết Chương 29: Thuật Toán Băm Xâu (Polynomial Rolling Hashing)

Lời giải & Đáp án: Bài 29.1 - Băm Chuỗi Đa Thức: Tiền Tố Băm, Rabin-Karp & Kiểm Tra Palindrome			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	abacaba aba 1 7	2 1 1	Tìm chuỗi mẫu đơn giản trong văn bản
2	racecar car 1 7	1 5 1	Chuỗi palindrome hoàn chỉnh
3	banana apple 2 6	0 -1 1	Không xuất hiện mẫu, kiểm tra đoạn con đối xứng
4	a a 1 1	1 1 1	Chuỗi chỉ 1 ký tự, mẫu 1 ký tự trùng
5	abc abcde 1 3	0 -1 0	Mẫu dài hơn chuỗi văn bản
6	abcdef cd 3 3	1 3 1	Đoạn con kiểm tra độ dài 1 luôn là palindrome
7	aaaaaa aa 1 6	5 1 1	Chuỗi lặp lại liên tục, mẫu gối nhau
8	noonnight night 1 4	1 5 1	Đoạn con dài chẵn đối xứng
9	radarad rad 1 5	2 1 1	Thử thách nhầm chuỗi 7 ký tự xen kẽ
10	abcabcba abc 3 6	2 1 0	Thử thách nhầm chuỗi 8 ký tự với nhiều vị trí khớp

Bảng Vết Thực Thi Từng Bước (Chương 29)

Bước	Mô tả / Biến	Giá trị
1	Read inputs S, P and interval [L, R]	-
-	n	7
-	m	3
-	q_l	1
-	q_r	7

Bước	Mô tả / Biến	Giá trị
2	Rabin-Karp search completed	-
-	occ_count	2
-	first_pos	1
3	Palindrome check completed	-
-	is_pal	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs S, P and interval [L, R]	-
-	n	7
-	m	3
-	q_l	1
-	q_r	7
2	Rabin-Karp search completed	-
-	occ_count	1
-	first_pos	5
3	Palindrome check completed	-
-	is_pal	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs S, P and interval [L, R]	-
-	n	6
-	m	5
-	q_l	2
-	q_r	6
2	Rabin-Karp search completed	-
-	occ_count	0
-	first_pos	-1
3	Palindrome check completed	-
-	is_pal	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs S, P and interval [L, R]	-
-	n	1
-	m	1
-	q_l	1
-	q_r	1
2	Rabin-Karp search completed	-
-	occ_count	1
-	first_pos	1
3	Palindrome check completed	-
-	is_pal	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs S, P and interval [L, R]	-
-	n	3
-	m	5
-	q_l	1
-	q_r	3
2	Rabin-Karp search completed	-
-	occ_count	0
-	first_pos	-1
3	Palindrome check completed	-
-	is_pal	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs S, P and interval [L, R]	-
-	n	6
-	m	2
-	q_l	3
-	q_r	3
2	Rabin-Karp search completed	-
-	occ_count	1
-	first_pos	3
3	Palindrome check completed	-
-	is_pal	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs S, P and interval [L, R]	-
-	n	6
-	m	2
-	q_l	1
-	q_r	6
2	Rabin-Karp search completed	-
-	occ_count	5
-	first_pos	1
3	Palindrome check completed	-
-	is_pal	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs S, P and interval [L, R]	-
-	n	9
-	m	5
-	q_l	1
-	q_r	4
2	Rabin-Karp search completed	-

Bước	Mô tả / Biến	Giá trị
-	occ_count	1
-	first_pos	5
3	Palindrome check completed	-
-	is_pal	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs S, P and interval [L, R]	-
-	n	7
-	m	3
-	q_l	1
-	q_r	5
2	Rabin-Karp search completed	-
-	occ_count	2
-	first_pos	1
3	Palindrome check completed	-
-	is_pal	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs S, P and interval [L, R]	-
-	n	8
-	m	3
-	q_l	3
-	q_r	6
2	Rabin-Karp search completed	-
-	occ_count	2
-	first_pos	1
3	Palindrome check completed	-
-	is_pal	0

Lời Giải Chi Tiết Chương 30: Thao Tác Bit & Quy Hoạch Động Trạng Thái Mặt Nạ (Bitmask DP)

Lời giải & Đáp án: Bài 30.1 - Thao Tác Bit & Quy Hoạch Động Trạng Thái Mặt Nạ: Bài Toán TSP			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	3 0 10 20 10 0 15 20 15 0	25 45	Tam giác 3 đỉnh chi phí đối xứng
2	4 0 2 9 10 1 0 6 4 15 7 0 8 6 3 12 0	16 21	Đồ thị 4 đỉnh chi phí tăng dần
3	3 0 1 5 5 0 2 1 5 0	3 4	Đồ thị 3 đỉnh có hướng một chiều rẽ
4	2 0 5 7 0	5 12	Cận dưới N=2 hai đỉnh
5	3 0 1 1 1 0 1 1 1 0	2 3	Đồ thị 3 đỉnh trọng số đều bằng 1
6	2 0 10 10 0	10 20	Đồ thị 2 đỉnh trọng số bằng nhau
7	4 0 1 20 20 20 0 1 20 20 20 0 1 1 20 20 0	3 4	Đồ thị 4 đỉnh với một chu trình rẽ vượt trội
8	4 0 3 4 5 3 0 5 4 4 5 0 3 5 4 3 0	10 14	Đồ thị 4 đỉnh ma trận đối xứng khoảng cách Manhattan
9	5 0 3 1 5 8 3 0 6 7 9 1 6 0 4 2 5 7 4 0 3 8 9 2 3 0	13 16	Thử thách nhăm TSP 5 đỉnh

Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
10	<pre> 6 0 2 4 6 8 10 2 0 3 5 7 9 4 3 0 2 4 6 6 5 2 0 3 5 8 7 4 3 0 2 10 9 6 5 2 0 </pre>	12 22	Thử thách nhầm TSP 6 đỉnh đối xứng

Bảng Vết Thực Thi Từng Bước (Chương 30)

Bước	Mô tả / Biến	Giá trị
1	Read input matrix N	-
-	n	3
2	Bitmask DP completed	-
-	min_path	25
-	min_tsp	45

Bước	Mô tả / Biến	Giá trị
1	Read input matrix N	-
-	n	4
2	Bitmask DP completed	-
-	min_path	16
-	min_tsp	21

Bước	Mô tả / Biến	Giá trị
1	Read input matrix N	-
-	n	3
2	Bitmask DP completed	-
-	min_path	3
-	min_tsp	4

Bước	Mô tả / Biến	Giá trị
1	Read input matrix N	-
-	n	2
2	Bitmask DP completed	-
-	min_path	5
-	min_tsp	12

Bước	Mô tả / Biến	Giá trị
1	Read input matrix N	-
-	n	3
2	Bitmask DP completed	-
-	min_path	2
-	min_tsp	3

Bước	Mô tả / Biến	Giá trị
1	Read input matrix N	-
-	n	2
2	Bitmask DP completed	-
-	min_path	10
-	min_tsp	20

Bước	Mô tả / Biến	Giá trị
1	Read input matrix N	-
-	n	4
2	Bitmask DP completed	-
-	min_path	3
-	min_tsp	4

Bước	Mô tả / Biến	Giá trị
1	Read input matrix N	-
-	n	4
2	Bitmask DP completed	-
-	min_path	10
-	min_tsp	14

Bước	Mô tả / Biến	Giá trị
1	Read input matrix N	-
-	n	5
2	Bitmask DP completed	-
-	min_path	13
-	min_tsp	16

Bước	Mô tả / Biến	Giá trị
1	Read input matrix N	-
-	n	6
2	Bitmask DP completed	-
-	min_path	12
-	min_tsp	22

Lời Giải Chi Tiết Bài Tập Tích Hợp Toàn Diện V1

Lời giải & Đáp án: Bài Tập Tích Hợp 4 - Hệ Thống Cây Vương Quốc: Cập Nhật Cây Con Euler Tour + Fenwick Tree & Khoảng Cách LCA			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	5 3 10 2 4 5 10 20 30 40 50 1 2 1 3 2 4 2 5	160 110 2 2	Cây 5 đỉnh, cập nhật đỉnh con và truy vấn LCA
2	4 1 5 3 1 4 5 10 15 20 1 2 2 3 3 4	55 35 1 3	Cây 4 đỉnh dạng chuỗi, cập nhật đỉnh gốc
3	5 4 100 4 2 5 1 2 3 4 5 1 2 1 3 1 4 1 5	115 104 1 2	Cây hình sao 5 đỉnh, cập nhật một nhánh lá
4	1 1 50 1 1 1 100	150 150 1 0	Cây 1 đỉnh duy nhất
5	2 2 0 2 1 2 30 40 1 2	70 40 1 1	Cây 2 đỉnh, cập nhật giá trị 0
6	3 2 15 2 2 2 10 20 30 1 2 1 3	75 35 2 0	Truy vấn LCA của hai đỉnh trùng nhau
7	6 3 -20 3 4 6 50 40 30 20 10 5 1 2 1 3 2 4 2 5 3 6	135 15 1 4	Cập nhật số âm làm giảm tài nguyên cây con
8	6 5 10 2 3 6 1 1 1 1 1 1 1 2 1 3 2 4 2 5 3 6	16 13 3 1	Cây 6 đỉnh với các đỉnh đều có giá trị ban đầu là 1

Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
9	7 4 12 3 6 7 1 2 3 4 5 6 7 1 2 1 3 2 4 2 5 3 6 3 7	40 16 3 2	Thử thách nhấm cây nhị phân 7 đỉnh đầy đủ
10	8 6 25 4 7 8 10 20 30 40 50 60 70 80 1 2 1 3 2 4 2 5 3 6 4 7 4 8	385 190 4 2	Thử thách nhấm cây 8 đỉnh phân nhánh sâu

Bảng Vết Thực Thi Từng Bước (Bài Tập Tích Hợp)

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree structure	-
-	n	5
-	p	3
-	x	10
-	q	2
-	u_query	4
-	v_query	5
2	Euler tour and binary lifting table constructed	-
-	tin	(0, 1, 2, 5, 3, 4)
-	tout	(0, 5, 4, 5, 3, 4)
3	Subtree sums and LCA query computed	-
-	s_root	160
-	s_q	110
-	lca_node	2
-	dist_uv	2

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree structure	-
-	n	4
-	p	1
-	x	5
-	q	3
-	u_query	1

Bước	Mô tả / Biến	Giá trị
-	v_query	4
2	Euler tour and binary lifting table constructed	-
-	tin	(0, 1, 2, 3, 4)
-	tout	(0, 4, 4, 4, 4)
3	Subtree sums and LCA query computed	-
-	s_root	55
-	s_q	35
-	lca_node	1
-	dist_uv	3

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree structure	-
-	n	5
-	p	4
-	x	100
-	q	4
-	u_query	2
-	v_query	5
2	Euler tour and binary lifting table constructed	-
-	tin	(0, 1, 2, 3, 4, 5)
-	tout	(0, 5, 2, 3, 4, 5)
3	Subtree sums and LCA query computed	-
-	s_root	115
-	s_q	104
-	lca_node	1
-	dist_uv	2

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree structure	-
-	n	1
-	p	1
-	x	50
-	q	1
-	u_query	1
-	v_query	1
2	Euler tour and binary lifting table constructed	-
-	tin	(0, 1)
-	tout	(0, 1)
3	Subtree sums and LCA query computed	-

Bước	Mô tả / Biến	Giá trị
-	s_root	150
-	s_q	150
-	lca_node	1
-	dist_uv	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree structure	-
-	n	2
-	p	2
-	x	0
-	q	2
-	u_query	1
-	v_query	2
2	Euler tour and binary lifting table constructed	-
-	tin	(0, 1, 2)
-	tout	(0, 2, 2)
3	Subtree sums and LCA query computed	-
-	s_root	70
-	s_q	40
-	lca_node	1
-	dist_uv	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree structure	-
-	n	3
-	p	2
-	x	15
-	q	2
-	u_query	2
-	v_query	2
2	Euler tour and binary lifting table constructed	-
-	tin	(0, 1, 2, 3)
-	tout	(0, 3, 2, 3)
3	Subtree sums and LCA query computed	-
-	s_root	75
-	s_q	35
-	lca_node	2
-	dist_uv	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree structure	-
-	n	6
-	p	3
-	x	-20
-	q	3
-	u_query	4
-	v_query	6
2	Euler tour and binary lifting table constructed	-
-	tin	(0, 1, 2, 5, 3, 4, 6)
-	tout	(0, 6, 4, 6, 3, 4, 6)
3	Subtree sums and LCA query computed	-
-	s_root	135
-	s_q	15
-	lca_node	1
-	dist_uv	4

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree structure	-
-	n	6
-	p	5
-	x	10
-	q	2
-	u_query	3
-	v_query	6
2	Euler tour and binary lifting table constructed	-
-	tin	(0, 1, 2, 5, 3, 4, 6)
-	tout	(0, 6, 4, 6, 3, 4, 6)
3	Subtree sums and LCA query computed	-
-	s_root	16
-	s_q	13
-	lca_node	3
-	dist_uv	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree structure	-
-	n	7
-	p	4
-	x	12
-	q	3
-	u_query	6

Bước	Mô tả / Biến	Giá trị
-	v_query	7
2	Euler tour and binary lifting table constructed	-
-	tin	(0, 1, 2, 5, 3, 4, 6, 7)
-	tout	(0, 7, 4, 7, 3, 4, 6, 7)
3	Subtree sums and LCA query computed	-
-	s_root	40
-	s_q	16
-	lca_node	3
-	dist_uv	2

Bước	Mô tả / Biến	Giá trị
1	Read inputs and tree structure	-
-	n	8
-	p	6
-	x	25
-	q	4
-	u_query	7
-	v_query	8
2	Euler tour and binary lifting table constructed	-
-	tin	(0, 1, 2, 7, 3, 6, 8, 4, 5)
-	tout	(0, 8, 6, 8, 5, 6, 8, 4, 5)
3	Subtree sums and LCA query computed	-
-	s_root	385
-	s_q	190
-	lca_node	4
-	dist_uv	2